In [41…
```python
# Q1. Student Record
# Create a dictionary named student containing the student's name, age, C

student = {
    "name" : "Saad Ali",
    "age" : 25,
    "CGPA" : 3.79,
    "department": "Software Engineering"
}

key = input("Enter a key: ")

if key in student:
    print(student[key])
else:
    print("Key Not Found")
```

```
Saad Ali
```

In [42…
```python
# Q2. Employee Information
# Create a dictionary named emp containing the following information:

# Name: Ali

# Position: HR

# Salary: 50000

# Department: IT

# Ask the user to enter a key and display its corresponding value. If the

emp = {
    "name": "Ali",
    "position": "HR",
    "salary": 50000,
    "department": "IT"
}

key = input("Enter a key: ")

if key in emp:
    print(emp[key])
else:
    print("Invalid Key")
```

```
50000
```

In [43…
```python
# Q3. Contact Book
# Create a dictionary containing the names and phone numbers of three con

contacts = {
    "Sara": "03001234567",
    "Ali": "03111234567",
```

```
    "Ahmad": "03211234567"
}

name = input("Enter contact name: ")

if name in contacts:
    print(contacts[name])
else:
    print("Contact Not Found")
```

```
03001234567
```

In [44…

```
# Q4. Nested Student Dictionary
# Create a nested dictionary named students containing the following reco

# Ali: Age 21, CGPA 3.7, Department CS

# Ahmad: Age 22, CGPA 3.4, Department AI

# Sara: Age 20, CGPA 3.9, Department SE

# Ask the user to enter a student's name and display their complete recor

students = {
    "Ali": {
        "age": 21,
        "cgpa": 3.7,
        "department": "CS"
    },
    "Ahmad": {
        "age": 22,
        "cgpa": 3.4,
        "department": "AI"
    },
    "Sara": {
        "age": 20,
        "cgpa": 3.9,
        "department": "SE"
    }
}

name = input("Enter student name: ")

if name in students:
    print(students[name])
else:
    print("Student Not Found")
```

```
{'age': 20, 'cgpa': 3.9, 'department': 'SE'}
```

In [45…

```
# Q5. Dictionary Methods
# Consider the following dictionary:
# python
# student = {
#     "name": "Ali",
```

```
#     "age": 23,
#     "department": "CS"
# }
# Write code to:

# Display all keys.

# Display all values.

# Display all key-value pairs.

# Add a new key "cgpa" with the value 3.5 using update().

student = {
    "name": "Ali",
    "age": 23,
    "department": "CS"
}

# 1. Display all keys
print(student.keys())

# 2. Display all values
print(student.values())

# 3. Display all key-value pairs
print(student.items())

# 4. Add cgpa using update()
student.update({"cgpa": 3.5})

print(student)
```

```
dict_keys(['name', 'age', 'department'])
dict_values(['Ali', 23, 'CS'])
dict_items([('name', 'Ali'), ('age', 23), ('department', 'CS')])
{'name': 'Ali', 'age': 23, 'department': 'CS', 'cgpa': 3.5}
```

In [46…

```
# Q6. Online Shopping System
# Create a nested dictionary named products containing the following prod

# Laptop: Price 120000, Quantity 5

# Mouse: Price 1500, Quantity 10

# Keyboard: Price 3000, Quantity 0

# Ask the user to enter a product name.

# If the product exists and its quantity is greater than 0, print "Produc

# If the quantity is 0, print "Out of Stock".

# If the product does not exist, print "Product Not Found".
```

```python
products = {
    "Laptop": {
        "price": 120000,
        "quantity": 5
    },
    "Mouse": {
        "price": 1500,
        "quantity": 10
    },
    "Keyboard": {
        "price": 3000,
        "quantity": 0
    }
}

name = input("Enter product name: ")

if name in products:
    if products[name]["quantity"] > 0:
        print("Product Added to Cart")
    else:
        print("Out of Stock")
else:
    print("Product Not Found")
```

```
Product Added to Cart
```

In [47…

```python
# Q1. Exploring Data
# Display the first 10 rows of the dataset.

import pandas as pd

df = pd.read_csv("Student_data_clean.csv")
df.head(10)
```

Out[47]:

	student_id	age	gender	study_hours_per_day	social_media_hours	netflix_hours	p
0	S1000	21.048994	Female	0	0.029056	4.799806	
1	S1001	20.496321	Female	ten	3.539930	3.400655	
2	S1002	20.496321	Male	ten	10.270434	1.780307	
3	S1003	25.350557	Female	ten	0.816828	5.861446	
4	S1004	17.937754	Female	ten	5.320886	0.000000	
5	S1005	23.193687	Male	7.2	0.762742	4.156124	
6	S1006	22.062970	Female	5.6	0.000000	0.000000	
7	S1007	20.496321	unknown	4.3	0.000000	3.373847	
8	S1008	20.496321	Female	ten	3.663144	6.903434	
9	S1009	23.669075	Female	4.8	5.199887	0.000000	

In [48…

```python
# Q2. Dataset Information
```

```
# Display the number of rows and columns in the dataset.

print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
```

```
Number of rows: 1153
Number of columns: 17
```

In [49…

```
# Q3. Column Selection
# Display only the student_id, age, and exam_score columns.

df[['student_id', 'age', 'exam_score']]
```

Out[49]:

	student_id	age	exam_score
0	S1000	21.048994	59.197075
1	S1001	20.496321	98.718319
2	S1002	20.496321	33.361684
3	S1003	25.350557	27.830878
4	S1004	17.937754	60.686418
...	...	...	...
1148	S1431	20.169749	31.321091
1149	NaN	22.671669	45.230354
1150	S1492	16.382979	70.239270
1151	S1979	20.676107	55.382948
1152	S1902	24.716932	67.190582

1153 rows × 3 columns

In [50…

```
# Q4. Missing Values
# Find the number of missing values in each column.

df.isnull().sum()
```

Out[50]:
```
student_id                      121
age                               0
gender                            0
study_hours_per_day               0
social_media_hours                0
netflix_hours                     0
part_time_job                     0
attendance_percentage             0
sleep_hours                     119
diet_quality                      0
exercise_frequency                0
parental_education_level          0
internet_quality                  0
mental_health_rating              0
extracurricular_participation     0
exam_score                        0
random_id                         0
dtype: int64
```

In [51…
```
# Q5. Data Cleaning
# Remove duplicate rows from the dataset and store the cleaned result in

df_cleaned = df.drop_duplicates()

df_cleaned
```

Out[51]:

	student_id	age	gender	study_hours_per_day	social_media_hours	netflix_hours
0	S1000	21.048994	Female	0	0.029056	4.799806
1	S1001	20.496321	Female	ten	3.539930	3.400655
2	S1002	20.496321	Male	ten	10.270434	1.780307
3	S1003	25.350557	Female	ten	0.816828	5.861446
4	S1004	17.937754	Female	ten	5.320886	0.000000
…	…	…	…	…	…	…
1148	S1431	20.169749	Female	0	0.147439	2.220364
1149	NaN	22.671669	Female	ten	1.463998	1.937920
1150	S1492	16.382979	Male	3.8	0.000000	0.000000
1151	S1979	20.676107	Male	1.3	1.741795	0.000000
1152	S1902	24.716932	Male	4.1	1.071961	4.197893

1150 rows × 17 columns

In [53…
```
# Q6. Handling Missing Values
# Fill missing values in the sleep_hours column with its median.

df['sleep_hours'] = df['sleep_hours'].fillna(
    df['sleep_hours'].median()
```

```
)
df.isnull().sum()
```

Out[53]:

```
student_id                          121
age                                   0
gender                                0
study_hours_per_day                   0
social_media_hours                    0
netflix_hours                         0
part_time_job                         0
attendance_percentage                 0
sleep_hours                           0
diet_quality                          0
exercise_frequency                    0
parental_education_level              0
internet_quality                      0
mental_health_rating                  0
extracurricular_participation         0
exam_score                            0
random_id                             0
dtype: int64
```

In [54…

```
# Q7. Data Type Conversion
# Convert the study_hours_per_day column to numeric values. Handle invali

df['study_hours_per_day'] = pd.to_numeric(
    df['study_hours_per_day'],
    errors='coerce'
)
print(df['study_hours_per_day'].dtype)
```

```
float64
```

In [55…

```
# Q8. Filtering Rows
# Display all students whose exam_score is greater than 80.

df[df['exam_score'] > 80]
```

Out[55]:

	student_id	age	gender	study_hours_per_day	social_media_hours	netflix_hour
1	S1001	20.496321	Female	NaN	3.539930	3.40065
5	S1005	23.193687	Male	7.2	0.762742	4.15612
6	S1006	22.062970	Female	5.6	0.000000	0.00000
7	S1007	20.496321	unknown	4.3	0.000000	3.37384
8	S1008	20.496321	Female	NaN	3.663144	6.90343
...	...	...	...	...	...	.
1130	S1404	26.059073	unknown	4.6	0.303310	5.11063
1131	S1406	20.380449	Female	3.6	6.380638	1.21942
1145	S1530	15.523021	Male	NaN	0.000000	2.87955
1146	S1455	19.711286	Male	8.3	6.992849	0.26449
1147	S1647	20.430457	Female	NaN	4.165038	1.93792

325 rows × 17 columns

In [56…

```
# Q9. Multiple Conditions
# Display students who study more than 5 hours per day and sleep for at l

df[
    (df['study_hours_per_day'] > 5) &
    (df['sleep_hours'] >= 7)
]
```

Out[56]:

	student_id	age	gender	study_hours_per_day	social_media_hours	netflix_hour
94	S1094	21.877130	Female	5.3	2.350669	3.23716
131	S1131	20.106261	Female	7.2	5.547043	0.00000
171	S1171	20.119481	Female	5.3	2.888899	0.51136
214	S1214	18.872441	Male	5.7	5.414765	0.53228
222	S1222	22.194291	Male	6.7	1.406480	0.00000
232	S1232	18.784415	Female	6.5	7.203870	1.93792
293	S1293	19.556450	Female	7.4	4.421131	0.00000
298	S1298	21.636060	Female	5.4	2.611274	1.93792
306	S1306	25.733835	Female	6.0	2.767127	1.69636
310	S1310	20.494998	other	5.3	0.000000	5.23260
334	S1334	20.880612	Female	5.8	6.435063	1.93792
356	NaN	17.486032	Male	5.8	1.764736	0.00000
359	NaN	20.496321	Male	6.3	0.000000	0.00000
366	S1366	27.348863	Male	6.3	1.823476	1.02284
367	S1367	11.433406	Male	5.8	2.611274	1.58540
398	S1398	23.301959	Male	5.7	0.000000	0.00000
455	S1455	22.270798	Male	8.3	5.932053	6.20709
464	S1464	16.754015	Male	6.3	6.329983	5.90112
467	S1467	15.598833	Male	5.4	0.000000	0.00000
500	S1500	18.955051	Male	5.5	2.168226	1.03192
514	S1514	15.250609	Female	5.9	4.547371	3.46187
517	S1517	19.427826	Female	5.8	1.814388	0.00000
547	S1547	18.172251	Male	6.3	11.044803	1.93792
595	S1595	17.976623	Male	5.6	3.463348	0.00000
606	S1606	22.397474	Male	6.8	2.439326	0.00000
640	S1640	20.155345	Female	6.4	4.651400	0.00000
652	S1652	20.496321	Female	5.9	2.611274	3.25075
667	S1667	16.663527	Female	5.6	1.716976	3.98473
715	S1715	23.733864	unknown	5.6	0.000000	2.00833
764	S1764	16.323436	Male	6.6	1.268070	0.41354
809	NaN	19.091784	unknown	5.2	0.000000	5.75977
874	S1874	16.940867	Male	5.1	2.731454	0.30708
877	S1874	16.940867	Male	5.1	2.731454	0.30708

	student_id	age	gender	study_hours_per_day	social_media_hours	netflix_hour
932	NaN	20.818138	Female	5.7	0.000000	1.93792
994	S1991	19.841801	Male	6.0	0.719352	11.65320
1025	S1488	15.447670	Male	5.3	0.000000	2.08379
1042	S1875	15.575249	Female	7.6	0.000000	1.93792
1080	S1569	27.826733	Male	6.3	0.000000	2.89188
1146	S1455	19.711286	Male	8.3	6.992849	0.26449

In [57…
```
# Q10. Using isin()
# Display students whose internet_quality is either Good or Average.

df[df['internet_quality'].isin(['Good', 'Average'])]
```

Out[57]:

	student_id	age	gender	study_hours_per_day	social_media_hours	netflix_hours
0	S1000	21.048994	Female	0.0	0.029056	4.799806
1	S1001	20.496321	Female	NaN	3.539930	3.400655
3	S1003	25.350557	Female	NaN	0.816828	5.861446
4	S1004	17.937754	Female	NaN	5.320886	0.000000
8	S1008	20.496321	Female	NaN	3.663144	6.903434
…	…	…	…	…	…	…
1147	S1647	20.430457	Female	NaN	4.165038	1.937920
1148	S1431	20.169749	Female	0.0	0.147439	2.220364
1149	NaN	22.671669	Female	NaN	1.463998	1.937920
1150	S1492	16.382979	Male	3.8	0.000000	0.000000
1152	S1902	24.716932	Male	4.1	1.071961	4.197893

713 rows × 17 columns

In [58…
```
# Q11. Sorting Data
# Sort the dataset by exam_score in descending order and display the firs

df.sort_values(
    by='exam_score',
    ascending=False
).head(10)
```

Out[58]:

	student_id	age	gender	study_hours_per_day	social_media_hours	netflix_hour
1126	S1318	15.486285	unknown	2.2000	2.703445	3.12170
5	S1005	23.193687	Male	7.2000	0.762742	4.15612
620	S1620	14.544716	Female	1.4000	2.485553	5.26770
638	S1638	19.219215	Female	NaN	0.000000	2.90977
56	S1056	20.496321	Male	2.5000	4.903691	0.00000
23	S1023	24.238106	Female	11.0276	2.010763	1.76479
950	S1947	23.106798	Female	4.5000	1.747211	8.53082
952	NaN	17.368829	Female	4.4000	2.450181	0.00000
649	S1649	26.679326	Female	4.8000	0.800206	4.48841
652	S1652	20.496321	Female	5.9000	2.611274	3.25075

In [59…

```
# Q12. Finding Top Students
# Display the top 5 students with the highest exam scores. Show their stu

df.nlargest(5, 'exam_score')[
    ['student_id', 'study_hours_per_day', 'exam_score']
]
```

Out[59]:

	student_id	study_hours_per_day	exam_score
5	S1005	7.2000	100.0
7	S1007	4.3000	100.0
9	S1009	4.8000	100.0
23	S1023	11.0276	100.0
56	S1056	2.5000	100.0

In [60…

```
# Q13. Creating a New Column
# Create a new column named total_screen_time by adding social_media_hour

df['total_screen_time'] = (
    df['social_media_hours'] + df['netflix_hours']
)

df[['social_media_hours', 'netflix_hours', 'total_screen_time']]
```

Out[60]:

	social_media_hours	netflix_hours	total_screen_time
0	0.029056	4.799806	4.828862
1	3.539930	3.400655	6.940585
2	10.270434	1.780307	12.050742
3	0.816828	5.861446	6.678273
4	5.320886	0.000000	5.320886
...	...	...	...
1148	0.147439	2.220364	2.367803
1149	1.463998	1.937920	3.401919
1150	0.000000	0.000000	0.000000
1151	1.741795	0.000000	1.741795
1152	1.071961	4.197893	5.269854

1153 rows × 3 columns

In [61…

```python
# Q14. Applying a Function
# Create a function that assigns grades using the following rules:

# Score 85 or above: A

# Score 70 to below 85: B

# Score 55 to below 70: C

# Score 40 to below 55: D

# Score below 40: F

# Apply this function to the exam_score column and store the results in a

def assign_grade(score):
    if score >= 85:
        return 'A'
    elif score >= 70:
        return 'B'
    elif score >= 55:
        return 'C'
    elif score >= 40:
        return 'D'
    else:
        return 'F'


df['grade'] = df['exam_score'].apply(assign_grade)

df[['exam_score', 'grade']]
```

Out[61]:

	exam_score	grade
0	59.197075	C
1	98.718319	A
2	33.361684	F
3	27.830878	F
4	60.686418	C
...	...	...
1148	31.321091	F
1149	45.230354	D
1150	70.239270	B
1151	55.382948	C
1152	67.190582	C

1153 rows × 2 columns

In [62…

```
# Q15. Creating Categories with pd.cut()
# Create a new column named age_group using the following age intervals:

# 15–18: 15-18

# Above 18 to 21: 19-21

# Above 21 to 24: 22-24

# Above 24 to 35: 25+

# Use pd.cut() to assign the appropriate category to each student.

df['age_group'] = pd.cut(
    df['age'],
    bins=[15, 18, 21, 24, 35],
    labels=['15-18', '19-21', '22-24', '25+'],
    include_lowest=True,
    right=True
)

df[['age', 'age_group']]
```

Out[62]:

	age	age_group
0	21.048994	22-24
1	20.496321	19-21
2	20.496321	19-21
3	25.350557	25+
4	17.937754	15-18
...	...	...
1148	20.169749	19-21
1149	22.671669	22-24
1150	16.382979	15-18
1151	20.676107	19-21
1152	24.716932	25+

1153 rows × 2 columns

In [ ]: